UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2024-07, page [2]

Unofficial model answer:

a) The machine that John von Neumann himself invented (or is credited with designing) is the IAS machine (Institute for Advanced Study computer), also known as the von Neumann machine. It was the prototype for the stored-program computer.

b) The two's complement numbering system was introduced to provide a simple and efficient way to represent signed integers in binary, allowing both positive and negative numbers to be represented without a separate sign bit. It simplifies arithmetic operations because addition and subtraction can be performed using the same hardware, and it eliminates the problem of having two representations for zero (as in sign-magnitude) and the need for end-around carry (as in one's complement).

c) Two memory contents that are allowable in the Von Neumann architecture are: (1) instructions (program code) and (2) data (operands and results). In the von Neumann architecture, both instructions and data are stored in the same memory space.

Marking points:

• *1 mark: (a) Correctly names the IAS machine (or von Neumann machine) as the machine John von Neumann invented.*

• *2 marks: (b) Explains that two's complement was introduced to represent signed numbers efficiently (1 mark) and to simplify arithmetic (e.g., unified addition/subtraction, no separate sign bit, single zero) (1 mark).*

• *2 marks: (c) Names two allowable memory contents: instructions and data (1 mark each).*

Model confidence: high.

Model note: The question asks for the machine that John von Neumann himself invented; historically, the IAS machine is the correct answer, though some may refer to it as the von Neumann machine. For part (c), the Von Neumann architecture allows both instructions and data in the same memory; other contents like addresses or constants are also possible, but instructions and data are the most fundamental.

QUESTION 2 (5 marks)

Source: CSC429-2024-07, page [2]

Unofficial model answer:

a) The successor of the Difference Engine was the Analytical Engine, also designed by Charles Babbage. It was a general-purpose programmable mechanical computer, whereas the Difference Engine was a special-purpose calculator.

b) The Difference Engine was a special-purpose mechanical calculator designed to automatically compute and print mathematical tables (e.g., for naval navigation) using the method of finite differences; it performed only addition and could not be programmed. The Mechanical Calculating Machine (as referred to in the context of early mechanical calculators, often associated with Babbage's later work or contemporaneous devices) was a more general-purpose device capable of performing the four basic arithmetic operations (addition, subtraction, multiplication, and division) and could be set up to perform different calculations, though still not fully programmable like the Analytical Engine. In essence, the Difference Engine was fixed-function for table generation, while the Mechanical Calculating Machine offered broader arithmetic capability.

c) The computer generation of the Difference Engine (early 19th century, often called the mechanical era or 'zeroth' generation) relied on mechanical technology: gears, levers, shafts, and hand-cranked operation. There were no electronic components; power was human or steam, and data was represented by physical positions

© Hak Cipta Universiti Teknologi MARA

of mechanical parts. This technology allowed only slow, limited-precision computation and was prone to mechanical wear and manufacturing difficulties.

Marking points:

• 1 mark: (a) Correctly identifies the Analytical Engine as the successor.

• 2 marks: (b) Describes Difference Engine as special-purpose/fixed-function for table generation using finite differences (1 mark) and contrasts with Mechanical Calculating Machine as more general-purpose performing four arithmetic operations (1 mark).

• 2 marks: (c) Explains mechanical technology (gears, levers, hand-crank) (1 mark) and notes absence of electronics/limited capability (1 mark).

Model confidence: medium.

Model note: The term 'Mechanical Calculating Machine' is not precisely defined in the question; it may refer to Babbage's Analytical Engine or to contemporaneous mechanical calculators. The answer assumes a general contrast between special-purpose and general-purpose mechanical calculators. The 'computer generation' of the Difference Engine is not a standard generation in the usual electronic computer generations; it is interpreted as the mechanical era.

QUESTION 3 (5 marks)

Source: CSC429-2023-07, page [2]

Unofficial model answer:

Subpart (a): To solve -48 + 68 using sign-and-magnitude, first represent each operand in 8-bit sign-magnitude form: +48 = 00110000 (sign 0, magnitude 48 = 00110000), -68 = 11000100 (sign 1, magnitude 68 = 01000100). Since the signs differ, subtract the smaller magnitude from the larger magnitude: 68 - 48 = 20 = 00010100. The result takes the sign of the operand with the larger magnitude, which is -68, so the sign bit is 1. Therefore -48 + 68 = 10010100 in sign-magnitude, which is -20 in decimal. (Equivalently, +68 + (-48) = +20; the workflow above is the standard sign-magnitude addition rule.)

Subpart (b): To calculate -96 - 15 using 1's and 2's complement, first note that -96 - 15 = -96 + (-15) = -111. Using 8-bit representation: +96 = 01100000, so -96 in 1's complement = 10011111 and in 2's complement = 10100000. +15 = 00001111, so -15 in 1's complement = 11110000 and in 2's complement = 11110001. For 1's complement addition: 10011111 + 11110000 = 1 10001111; end-around carry gives 10001111 + 1 = 10010000, which is the 1's complement representation of -111 (since 01101111 = 111). For 2's complement addition: 10100000 + 11110001 = 1 10010001; discard the carry out, giving 10010001, which is the 2's complement representation of -111 (since 01101111 = 111). Thus -96 - 15 = -111 in both methods.

Marking points:

• 1 mark: Correct 8-bit sign-magnitude representations of +48 (00110000) and -68 (11000100) in part (a).

• 1 mark: Correct sign-magnitude subtraction of magnitudes (68 - 48 = 20) and assignment of negative sign, giving 10010100 = -20 in part (a).

• 1 mark: Correct 8-bit 1's complement representations of -96 (10011111) and -15 (11110000) and correct addition with end-around carry yielding 10010000 = -111 in part (b).

• 1 mark: Correct 8-bit 2's complement representations of -96 (10100000) and -15 (11110001) and correct addition with carry discarded yielding 10010001 = -111 in part (b).

• 1 mark: Clear complete workflow shown for both parts, including conversion to binary, operation, and conversion back to decimal.

Model confidence: high.

Model note: Assumes 8-bit word length, which is sufficient for the given ranges. The question text uses 'I's complement' meaning 1's complement. No figure or table was supplied; the answer is based solely on the question text. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2017-07, page [4]

© Hak Cipta Universiti Teknologi MARA

Unofficial model answer:

a) Cache: A small, high-speed memory placed between the CPU and main memory that holds recently or frequently accessed instructions and data, so that the CPU can retrieve them quickly and reduce the average time needed to access memory. It exploits locality of reference (temporal and spatial) and is usually built from SRAM.

b) Cache vs main memory:

i) Price: Cache is much more expensive per bit than main memory because it uses faster SRAM technology and is closer to the CPU; main memory (DRAM) is cheaper per bit.

ii) Memory size: Cache is much smaller in capacity (typically kilobytes to a few megabytes) than main memory, which is much larger (typically gigabytes).

iii) Access time: Cache has a much shorter access time (a few nanoseconds or less) than main memory, whose access time is longer (tens to hundreds of nanoseconds).

iv) Distance from CPU: Cache is physically closer to the CPU (often on the same chip or in the CPU package), whereas main memory is farther away, connected via the memory bus.

Marking points:

• 1 mark: Correct definition of cache as a small, fast memory between CPU and main memory that stores recently/frequently used data to reduce average access time (mention of locality or SRAM acceptable).

• 1 mark: Price — cache is more expensive per bit than main memory (or main memory is cheaper per bit).

• 1 mark: Memory size — cache is smaller than main memory (or main memory is larger).

• 1 mark: Access time — cache has shorter/faster access time than main memory.

• 1 mark: Distance from CPU — cache is closer to the CPU than main memory.

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. No figure or table was supplied; the answer is based solely on the question text. Marks are allocated as 1 mark for part (a) and 1 mark for each of the four characteristics in part (b), totaling 5 marks.

QUESTION 5 (10 marks)

Source: CSC429-2017-07, page [3]

Unofficial model answer:

a) A computer bus is a shared communication pathway that transfers data, addresses, and control signals between components. The address bus carries the memory or I/O address generated by the CPU to select a specific location; it is typically unidirectional (CPU to memory/I/O) and its width determines the maximum addressable memory (e.g., 32 address lines → 2^32 = 4 GiB). The data bus carries the actual data between the CPU, memory, and I/O devices; it is bidirectional and its width determines the number of bits transferred per cycle (e.g., 64-bit data bus). The control bus carries timing and command signals such as read/write, memory/IO select, interrupt request, clock, and bus grant/request; it coordinates and synchronizes the other buses. (Some texts also distinguish a system bus, which combines address, data, and control lines, and an expansion/I/O bus such as PCI/PCIe/USB that connects peripherals to the system.)

b) Synchronous bus timing uses a common clock so that all bus transactions occur at fixed, predetermined time intervals. Benefit 1: Simplicity and low cost — because events are referenced to the clock, the bus protocol and control logic are simpler (fewer handshake lines and less combinational logic), reducing design complexity and cost. Benefit 2: Higher speed/faster transfer for devices that can match the clock — fixed clock cycles allow predictable, high-throughput block transfers and easier pipelining, since no time is wasted waiting for asynchronous handshake acknowledgements. (A further benefit is easier interfacing of devices that share the same clock.)

Marking points:

• 1 mark: Address bus — carries address from CPU to memory/I/O; unidirectional; width determines addressable

© Hak Cipta Universiti Teknologi MARA

memory.

• 1 mark: Data bus — carries data between CPU, memory and I/O; bidirectional; width determines transfer size.

• 1 mark: Control bus — carries control/timing signals (read/write, memory/IO select, interrupt, clock) to coordinate transfers.

• 1 mark: Correctly identifies system bus / expansion (I/O) bus distinction or gives a valid additional bus type with function.

• 1 mark: Synchronous timing defined as using a common clock with fixed, predetermined transaction intervals.

• 1 mark: Benefit 1 stated (simplicity/low cost) with explanation (fewer handshake lines, simpler control logic).

• 1 mark: Benefit 2 stated (higher speed/predictable throughput) with explanation (fixed cycles, no waiting for acknowledgements, easier pipelining).

• 1 mark: Each benefit must be discussed (not merely listed) — up to 2 marks for discussion quality.

• 1 mark: Use of correct terminology (unidirectional/bidirectional, addressable memory, handshake, clock cycle).

• 1 mark: Overall coherence and accurate comparison of bus types.

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. The question text does not specify which bus types are expected, so the answer covers the classic address/data/control buses plus system/expansion bus. If the course taught a different taxonomy (e.g., processor-memory bus vs. I/O bus), adapt accordingly. Mark allocation is indicative and should be adjusted to the lecturer's rubric.

QUESTION 6 (10 marks)

Source: CSC429-2023-02, page [5]

Unofficial model answer:

a) i) MIMD with shared memory: Draw several CUs (CU1, CU2, ... CUn), each with its own Instruction Stream (IS1, IS2, ... ISn) entering it from above. Each CU drives its own PU (PU1...PUn). All PUs connect through a single common interconnection network (e.g. a bus) to one shared MU (or shared memory block). The Data Streams (DS) are drawn as double-headed arrows between each PU and the shared MU via the interconnection network, because data flows both ways (read and write). Label the shared memory 'MU (shared)' and label each IS and DS arrow. Key point: one memory space, multiple independent instruction streams, multiple data streams.

a) ii) MIMD with distributed memory: Draw n independent nodes. Each node contains its own CU, its own PU and its own local MU (CU1+PU1+MU1, CU2+PU2+MU2, ... CUn+PUn+MUn). Each CU receives its own Instruction Stream (IS1...ISn). Each PU has a Data Stream (DS) to its own local MU. The nodes are joined by an interconnection network (e.g. a message-passing network) drawn between the PUs/nodes. Key point: each processor has a private memory; no single shared MU; processors communicate by messages over the network.

b) i) MIMD with shared memory example: a symmetric multiprocessor (SMP) server/workstation or a multicore CPU where several cores share one main memory (e.g. a dual/quad-core PC with a common RAM and cache-coherence protocol).

b) ii) MIMD with distributed memory example: a cluster of networked computers/Beowulf cluster, or a massively parallel processor such as a distributed-memory supercomputer where each node has its own memory and nodes exchange messages (e.g. via MPI).

c) Advantage of MIMD with distributed memory: scalability. Because each processor has its own private local memory, adding more nodes adds both processing power and memory bandwidth without all processors contending for one shared bus/memory; this avoids the shared-memory bottleneck and allows the system to grow to very large numbers of processors (and each processor accesses its local memory quickly).

Marking points:

• a) i) 3 marks: correct set of multiple CUs each with its own labelled IS (1), multiple PUs each with its own DS (1), all connected to one shared MU via an interconnection network with IS/DS clearly labelled (1).

© Hak Cipta Universiti Teknologi MARA

• a) ii) 3 marks: n nodes each with own CU, PU and local MU (1), separate labelled IS per CU and DS between PU and local MU (1), interconnection network linking the nodes shown and labelled (1).

• b) i) 1 mark: correct shared-memory MIMD example (e.g. SMP/multicore with common RAM).

• b) ii) 1 mark: correct distributed-memory MIMD example (e.g. cluster/distributed-memory supercomputer).

• c) 2 marks: states scalability/no shared-memory bottleneck (1) with a brief justification that private local memory avoids contention and allows growth (1).

Model confidence: high.

Model note: Figure 3 is supplied only as a caption/crop description (SISD with CU, PU, MU and IS/DS arrows); the diagrams in (a) are described in words because a drawing cannot be rendered in JSON. Marking is an unofficial model allocation, not an official UTM scheme.

QUESTION 7 (10 marks)

Source: CSC429-2025-07, page [4]

Unofficial model answer:

a) The CPU must be faster than all I/O devices because it executes instructions and coordinates the entire system; if I/O devices were as fast as or faster than the CPU, the CPU would be overwhelmed and unable to service them in a timely manner. The CPU's high speed allows it to poll, interrupt, and manage multiple slower peripherals, and to perform useful computation while I/O transfers proceed asynchronously. In short, the CPU is the master that must not become the bottleneck; its speed ensures it can keep up with control and data-transfer demands from all devices.

b) I/O controllers act as intermediaries between the CPU and I/O devices. They provide a standard interface (registers, control lines) that the CPU can read/write, translate CPU commands into device-specific signals, buffer data to reconcile speed differences, and manage synchronization (status flags, interrupts). The CPU issues commands to the controller (e.g., read/write, device address), the controller performs the transfer with the device, and then signals completion via status bits or an interrupt. This decouples the CPU from device-specific timing and electrical details.

c) Two types of I/O modules: (1) Programmed I/O module – the CPU directly controls the transfer by polling status and moving data word by word; the module is simple but ties up the CPU. (2) Interrupt-driven I/O module – the module interrupts the CPU when a device is ready or a transfer completes, allowing the CPU to do other work; it is more efficient than polling. (Other acceptable types: DMA module, channel I/O module.)

d) An I/O controller for a specific device would be installed when a new peripheral is added to the system, when the existing controller fails or is upgraded, or when the device's interface standard differs from what the current controller supports. It is also installed during system configuration to match the device's address, interrupt line, and data format.

Marking points:

• 1 mark: CPU must be faster to avoid becoming bottleneck / to service slower devices

• 1 mark: CPU speed allows polling/interrupt handling and asynchronous operation

• 1 mark: I/O controller provides standard interface/registers for CPU

• 1 mark: controller translates commands and buffers data / handles synchronization

• 1 mark: controller signals completion via status/interrupt

• 1 mark: programmed I/O module described (CPU polls and transfers)

• 1 mark: interrupt-driven I/O module described (interrupt on ready/completion)

• 1 mark: comparison/efficiency noted

• 1 mark: installation when new device added or controller fails/upgraded

• 1 mark: installation when interface mismatch or system configuration

Model confidence: high.

Model note: No figure or table supplied; answer based on standard computer organization concepts. Part c accepts any two valid I/O module types (programmed, interrupt-driven, DMA, channel).

© Hak Cipta Universiti Teknologi MARA

QUESTION 8 (10 marks)

Source: CSC429-2018-12, page [3]

Unofficial model answer:

a) To convert 123.6875_{10} to IEEE 754 single precision: The number is positive, so sign bit S = 0. Convert the integer part: $123_{10} = 1111011_2$. Convert the fractional part: 0.6875 × 2 = 1.375 → 1; 0.375 × 2 = 0.75 → 0; 0.75 × 2 = 1.5 → 1; 0.5 × 2 = 1.0 → 1, so $0.6875_{10} = 0.1011_2$. Thus $123.6875_{10} = 1111011.1011_2$. Normalize: $1.1110111011_2 \times 2^6$. The exponent field is E = 6 + 127 = $133_{10} = 10000101_2$. The fraction field is the 23 bits after the leading 1: 1110111011 followed by 13 zeros = $11101110110000000000000_2$. Grouping into 4-bit nibbles: 0100 0010 1111 0111 0100 0000 0000 $0000_2 = 42F74000_{16}$. Therefore, the IEEE single precision hexadecimal representation is 42F74000H.

b) To convert $C24CC000_{16}$ to decimal: Expand to binary: C=1100, 2=0010, 4=0100, C=1100, C=1100, 0=0000, 0=0000, 0=0000, giving 1100 0010 0100 1100 1100 0000 0000 0000_2. The sign bit S = 1 (negative). The exponent field is $10000100_2 = 132_{10}$, so the unbiased exponent e = 132 − 127 = 5. The fraction field is $10011001100000000000000_2$. The significand is 1.100110011_2. Thus the value is $-1.100110011_2 \times 2^5 = -110011.0011_2$. Converting to decimal: $110011_2 = 51$, and $0.0011_2 = 1/8 + 1/16 = 0.1875$, so the magnitude is 51.1875. Therefore, C24CC000H represents -51.1875_{10}.

Marking points:

- *1 mark: Correct sign bit S = 0 for part (a).*
- *1 mark: Correct binary conversion of integer part $123_{10} = 1111011_2$.*
- *1 mark: Correct binary conversion of fractional part $0.6875_{10} = 0.1011_2$.*
- *1 mark: Correct normalization $1.1110111011_2 \times 2^6$ and exponent field $E = 133_{10} = 10000101_2$.*
- *1 mark: Correct 23-bit fraction field and final hexadecimal answer 42F74000H for part (a).*
- *1 mark: Correct binary expansion of C24CC000H and identification of sign bit S = 1 (negative).*
- *1 mark: Correct exponent field $10000100_2 = 132_{10}$ and unbiased exponent e = 5.*
- *1 mark: Correct significand 1.100110011_2 and value $-1.100110011_2 \times 2^5$.*
- *1 mark: Correct conversion of integer part $110011_2 = 51$.*
- *1 mark: Correct conversion of fractional part $0.0011_2 = 0.1875$ and final decimal answer -51.1875_{10}.*

Model confidence: high.

Model note: The question does not specify rounding mode; the conversion is exact, so no rounding is needed. The hexadecimal answers are given without the '0x' prefix, using the 'H' suffix as in the question. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2018-01, page [3]

Unofficial model answer:

a) i) The size of the data block is 64 bits (8 bytes). Justification: the data bus width determines how many bits can be transferred in one bus transaction/clock cycle, so a 64-bit data bus moves a 64-bit block per transfer. The 16-bit address bus does not change the block size; it only determines how many such locations (2^16 = 65,536 addressable locations) can be selected. Thus each addressed location holds one 64-bit word, giving a data block of 64 bits.

a) ii) The address bus is used to carry the address of the memory location or I/O port that the CPU wishes to access. It is unidirectional (from CPU/master outward), and its width fixes the size of the addressable memory space: with 16 bits, 2^16 = 65,536 distinct addresses (64 KiB if byte-addressed) can be selected. It thus selects which device/location participates in the data transfer.

a) iii) If a wider address bus is set, the number of addressable locations increases as 2^n (e.g., 16 bits → 65,536 locations; 20 bits → 1,048,576; 32 bits → 4 Gi addresses). This allows the CPU to address a larger

© Hak Cipta Universiti Teknologi MARA

memory and more I/O devices, but it requires wider address registers, wider address lines/pins, and more complex decoding; it does not by itself increase the data block size, which is set by the data bus.

b) Two types of bus timing: (1) Synchronous bus timing — all bus events are tied to a common clock; transfers occur on fixed clock edges, so it is simple, fast and cheap but every device must operate at the clock rate and the bus must be designed for the slowest device (or wait states added). (2) Asynchronous bus timing — no common clock; a handshake (e.g., request/acknowledge or strobe/acknowledge) coordinates the master and slave, so devices of different speeds can interoperate and the transfer takes as long as needed, at the cost of extra control lines and more complex logic. (A third, semi-synchronous, is a hybrid but only two are required.)

Marking points:

- *1 mark: (a)(i) states data block = 64 bits / 8 bytes*
- *1 mark: (a)(i) justification that data bus width sets bits transferred per transaction, address bus only selects location*
- *1 mark: (a)(ii) address bus carries address of memory location/I/O port to be accessed*
- *1 mark: (a)(ii) width determines addressable space, 2^16 = 65,536 locations*
- *1 mark: (a)(iii) wider address bus increases addressable locations as 2^n / larger memory and I/O space*
- *1 mark: (a)(iii) notes need for wider registers/pins/decoding and that data block size is unchanged*
- *2 marks: (b) synchronous timing described (common clock, fixed edges) with one advantage/limitation*
- *2 marks: (b) asynchronous timing described (handshake, no common clock) with one advantage/limitation*
- *1 mark: overall clarity/correct terminology (unidirectional address bus, handshake, wait states)*

Model confidence: high.

Model note: No figure/table was supplied; answer is based on standard bus-width definitions. Assumes byte-addressable memory when quoting 64 KiB for a 16-bit address bus; if word-addressed, the addressable count is still 2^16 locations. Mark allocation is an unofficial suggested split of the 10 marks.

QUESTION 10 (10 marks)

Source: CSC429-2013-12, page [2]

Unofficial model answer:

The table is best completed by matching each generation to its characteristic technology and a representative machine/inventor. (i) Zero generation technology: mechanical (already given); the machine listed is the Difference Engine (Charles Babbage), and the ABC (Atanasoff–Berry Computer) is also a zero-generation electronic-mechanical special-purpose machine. (ii) The inventor paired with the ABC is John V. Atanasoff and Clifford Berry (already given). (iii) Gottfried von Leibniz is associated with the stepped Reckoner (Leibniz wheel), a mechanical calculator of the zero/pre-generation era. (iv) First-generation technology: vacuum tubes (thermionic valves). (v) First-generation machine: ENIAC (Electronic Numerical Integrator and Computer), by John W. Mauchly and J. Presper Eckert (the table's 'EMAC' is a typo for ENIAC). (vi) The machine by Maurice Wilkes is EDSAC (Electronic Delay Storage Automatic Calculator), the first stored-program computer using mercury delay-line memory. (vii) Second-generation technology: transistors. (viii) Third-generation machine: IBM System/360 family (representative of integrated-circuit mainframes); Jack Kilby is credited with the integrated circuit. (ix) Fourth-generation technology: microprocessors / VLSI (large-scale integration). (x) Fifth-generation technology: artificial intelligence / parallel processing (ULSI, knowledge-based systems).

Marking points:

- *1 mark: (i) Difference Engine / mechanical calculator (Babbage) — zero generation machine*
- *1 mark: (ii) Atanasoff–Berry Computer (ABC) — already stated; accept John V. Atanasoff and Clifford Berry*
- *1 mark: (iii) Leibniz stepped Reckoner / mechanical calculator (Leibniz wheel)*
- *1 mark: (iv) Vacuum tubes (thermionic valves) — first-generation technology*
- *1 mark: (v) ENIAC — first-generation machine (Mauchly and Eckert)*
- *1 mark: (vi) EDSAC — Wilkes' stored-program machine*
- *1 mark: (vii) Transistors — second-generation technology*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: (viii) IBM System/360 (or representative IC mainframe) — third-generation machine

• 1 mark: (ix) Microprocessors / VLSI — fourth-generation technology

• 1 mark: (x) Artificial intelligence / parallel processing (ULSI) — fifth-generation technology

Model confidence: medium.

Model note: The extracted table text is garbled ('Ceperatlon', 'Anatasoft', 'EMAC'); ENIAC is assumed for 'EMAC'. Some cells (e.g., zero-generation machine, third-generation machine) are ambiguous because the table mixes technology, machine and inventor columns; answers reflect standard CSC429 generation classifications. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-02, page [6]

Unofficial model answer:

The equation in Figure 2 is interpreted as y = A - (2C * D) / (B + A^2). Registers A, B, C, D are fixed; x, y, z are temporary; final result must be in y. Table 1 provides one-address (LOADp, STORp, ADDp, SUBp, MULp, DIVp), two-address (MOV(p,q), ADD(p,q), SUB(p,q), MUL(p,q), DIV(p,q)), and three-address (ADD(p,q,r), SUB(p,q,r), MUL(p,q,r), DIV(p,q,r)) instructions. In two- and three-address notations, the first operand is the destination. For one-address, the accumulator is implicit; we use x as accumulator and y as final storage.

One-address program: LOAD A; MUL A; STOR z; LOAD B; ADD z; STOR z; LOAD C; MUL C; MUL D; STOR x; LOAD A; SUB x; STOR y; LOAD x; DIV z; STOR y. Explanation: compute A^2 into z, then B+A^2 into z; compute 2C*D into x; compute A - x into y; finally divide by z and store in y. This uses only the listed one-address instructions and temporary registers x, y, z.

Two-address program: MOV(x, A); MUL(x, A); ADD(x, B); MOV(z, C); MUL(z, C); MUL(z, D); MOV(y, A); SUB(y, z); DIV(y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the two-address column.

Three-address program: MUL(x, A, A); ADD(x, x, B); MUL(z, C, C); MUL(z, z, D); SUB(y, A, z); DIV(y, y, x). Explanation: x = A^2 + B; z = 2C*D; y = A - z; y = y / x. The final result is in y. All instructions are from the three-address column.

Note: The figure was supplied only as an image asset; the equation is inferred from the extracted text 'y = A - 2C * D / (B + A2)'. If the intended grouping differs (e.g., (2C*D)/(B+A^2) versus 2C*(D/(B+A^2))), the instruction sequences would change accordingly. The answer assumes standard operator precedence and left-to-right evaluation for multiplication/division.

Marking points:

*• 1 mark: Correctly interpret the equation as y = A - (2C * D) / (B + A^2) or equivalent, showing understanding of operator precedence.*

• 1 mark: State that one-address instructions use an implicit accumulator and that x, y, z are used as temporary/final storage.

• 2 marks: One-address program correctly computes A^2 and B+A^2 using LOAD, MUL, ADD, STOR (partial credit for minor sequencing errors).

*• 2 marks: One-address program correctly computes 2C*D using LOAD, MUL, MUL, STOR.*

*• 2 marks: One-address program correctly computes A - (2C*D) and divides by (B+A^2), storing final result in y.*

• 1 mark: Two-address program correctly computes x = A^2 + B using MOV, MUL, ADD.

*• 1 mark: Two-address program correctly computes z = 2C*D using MOV, MUL, MUL.*

• 2 marks: Two-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: Three-address program correctly computes x = A^2 + B using MUL, ADD.

*• 1 mark: Three-address program correctly computes z = 2C*D using MUL, MUL.*

• 2 marks: Three-address program correctly computes y = A - z and y = y / x, final result in y.

• 1 mark: All instructions used are from Table 1 and correct assembly notation (operand order, register names) is followed.

• 1 mark: Overall clarity, correct use of temporary registers, and final result stored in y for all three programs.

• 1 mark: Acknowledges any ambiguity in the figure/equation (e.g., grouping) and states assumptions.

Model confidence: medium.

*Model note: The equation was inferred from the extracted text 'y = A - 2C * D / (B + A2)' because the figure was provided only as an image asset without a machine-readable equation. The answer assumes standard precedence: multiplication and division before subtraction, and left-to-right for * and /. If the intended equation is different (e.g., y = A - 2C * (D/(B+A^2))), the programs would*

© Hak Cipta Universiti Teknologi MARA

need adjustment. Table 1 was provided as a native table and is used as-is. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA